

AlgoInf Lernzettel

Lil'Jana

August 10, 2026

Contents

1	Einführung	3
2	Datenstrukturen	4
2.1	Aufgabe: Abstrakte Datentypen zuweisen	4
2.2	Adjazenzmatrix und Kantenliste	5
2.2.1	Aufgabe: Löschen und Einfügen ✓	5
2.2.2	Aufgabe: Speicheraufwand, Nachfolger finden und Einfluss auf die asymptotische Laufzeit ✓	6
2.2.3	Aufgabe: Einfluss auf den Aufwand von Floyd ✓	7
2.2.4	Aufgabe: Einfluss auf asymptotischen Aufwand von Floyd ✓	8
2.3	Eulerkreisalgorithmus	9
2.3.1	Aufgabe: Einfluss auf den Aufwand vom Eulerkreisalgorithmus ✓	10
2.3.2	Aufgabe: Eignung bezüglich Grapheneigenschaften ✓	11
2.4	Queue	12
2.4.1	Aufgabe: Breitensuche ✓	13
2.4.2	Aufgabe: Realisierung einer Priority Queue ✓	14
2.4.3	Aufgabe: Realisierung Indexed Priority Queue ✓	15
2.4.4	Einfluss der Realisierung auf den Aufwand von Dijkstra ✓	16
3	Algorithmen	17
3.0.1	Aufgabe: Algorithmus für negative Kantengewichte ✓	18
3.1	Dijkstra	19
3.1.1	Aufgabe: Eigenschaft zum Zeigen der Korrektheit ✓	20
3.2	Lineare Probleme	21
3.2.1	Aufgabe: Relaxation eines ILP ✓	22
3.2.2	Aufgabe: Lösung einer Relation ✓	23
3.2.3	Aufgabe: Relation zwischen optimalen Werten ✓	24
3.2.4	MAXSAT zu ILP I. ✓	25
3.2.5	MAXSAT zu ILP II. ✓	26
3.3	Floyd-Warshall	27
3.3.1	Eigenschaft der Distanzmatrix ✓	27
3.4	Weighted Vertex Cover Problem ✓	28
3.4.1	Aufgabe: Algorithmus zur Approximation ✓	29
3.5	Kruskal	30
3.5.1	Programmiereparadigma von Kruskal	30
4	Anwendung	31
4.1	Graph 1	32
4.1.1	Finde Eulerpfad bzw. Eulerkreis ✓	32
4.1.2	Änderung der Distanzmatrix ✓	32
4.1.3	Der Graph als Flussnetzwerk	32
4.1.4	Optimaler Fluss	33
4.2	Graph 2	34
4.2.1	Finde Eulerpfad bzw. Eulerkreis	34
4.2.2	Ergebnis von Kruskal	34
4.2.3	Kürzeste Pfade Baum	34
4.2.4	Der Graph als Flussnetzwerk	34

5	Linear Programming	35
5.1	Verwandeln Sie folgendes LP in ein LP in Standardform	36
5.2	Gegeben sei folgendes LP in Schlupf-Form . Was ist die Basislösung? Ist diese zulässig? Warum/Warum nicht?	38
5.3	Simplex-Verfahren	41
5.3.1	Führen Sie in folgendem LP einen Pivotschritt durch:	41
5.4	Formen Sie folgendes LP um, so dass es eine gültige Basislösung hat und geben Sie diese an! . .	45
5.5	Besitzt folgendes LP eine zulässige Lösung? Warum / warum nicht?	46
6	Multiple Choice	47
	Index	50
	List of Algorithms	54
	References	54

1 Einführung

Das Dokument ist geordnet nach den Aufgabentypen in Altklausuren, mit entsprechenden Titeln.
Es wurde ChatGPT OpenAI 2024 u.a. für Zusammenfassungen zur Hilfe genommen.

2 Datenstrukturen

2.1 Aufgabe: Abstrakte Datentypen zuweisen

Geben Sie für die folgenden Algorithmen aus der Vorlesung jeweils einen abstrakten Datentypen an (abgesehen von Graph), auf dem der Algorithmus basiert: Tiefensuche , Eulerkreis , Prim ! *Klausur Algorithmen der Informatik Sommersemester 2021 2021*

2.2 Adjazenzmatrix und Kantenliste

2.2.1 Aufgabe: Löschen und Einfügen ✓

Nennen Sie für die Kantenliste und die Adjazenzmatrix jeweils den Aufwand zum Löschen einer bestimmten Kante (u, v) und für das Hinzufügen einer neuen Kante. Erklären Sie jeweils, wie diese zustande kommen! *Probeklausur Algorithmen der Informatik SoSe 2024 2024 Probeklausur Algorithmen der Informatik SS 2021 2021*

Datenstruktur	Aktion	Antwort	Erklärung
Adjazenzmatrix	löschen	$O(1)$	Um eine Kante zu löschen, setzt man einfach den Wert in der Matrix an der Position auf 0. Dies ist eine konstante Zeitoperation, da der Zugriff auf ein Element in einer Matrix konstanten Aufwand hat.
	hinzufügen	$O(1)$	Um eine Kante zu löschen, setzt man einfach den Wert in der Matrix an der Position auf 1. Dies ist eine konstante Zeitoperation, da der Zugriff auf ein Element in einer Matrix konstanten Aufwand hat.
Kantenliste	löschen	$O(E)$	Um eine Kante zu löschen, müssen wir die Kantenliste durchsuchen, um die Kante zu finden und zu entfernen. Im schlimmsten Fall muss man jede Kante in der Liste überprüfen
	hinzufügen	$O(1)$	Konstanter Aufwand

2.2.2 Aufgabe: Speicheraufwand, Nachfolger finden und Einfluss auf die asymptotische Laufzeit ✓

Nennen Sie für die Kantenliste und die Adjazenzmatrix jeweils den Aufwand zum Speichern eines Graphen und für das Ausgeben eines Nachfolgers für einen Knoten u . Erklären Sie jeweils, wie diese zustande kommen!

Nennen Sie zudem einen Graphen-Algorithmus, bei dem die Verwendung dieser beiden Graphen-Speicherstrukturen zu unterschiedlichen asymptotischen Laufzeiten führt und erläutern Sie, warum (etwa durch Benennung der den Laufzeitunterschied verursachenden, charakteristischen Operationen) *Klausur Algorithmen der Informatik Sommersemester 2021 2021*

Datenstruktur	Eigenschaft	Antwort	Erklärung
Kantenliste	Speicheraufwand	$O(E)$	Die Kantenliste speichert jede Kante einmal, daher ist der Speicheraufwand direkt proportional zur Anzahl der Kanten.
	Ausgabe Nachfolger	$O(E)$	Um alle Nachfolger eines Knotens u zu finden, muss man die gesamte Kantenliste durchgehen und alle Kanten prüfen. Im schlimmsten Fall muss man jede Kante betrachten.
Adjazenzmatrix	Speicheraufwand	$O(n^2)$	Eine Adjazenzmatrix benötigt Speicherplatz für alle möglichen Kanten zwischen n Knoten, was zu einer Matrix der Größe $n \times n$ führt.
	Ausgabe Nachfolger	$O(n)$	Um alle Nachfolger eines Knotens zu finden, muss man die Zeile u der Matrix durchgehen und alle Einträge prüfen. Da es n mögliche Einträge gibt, ist der Aufwand linear in Bezug auf die Anzahl der Knoten.

Bei 's Algorithmus führt die Verwendung dieser beiden Graphen- Speicherstrukturen zu unterschiedlichen asymptotischen Laufzeiten.

Datenstruktur	Laufzeit	Erklärung
Kantenliste	$O(E \log n)$	Bei der Kantenliste muss der Algorithmus alle Kanten durchsuchen, um die Nachbarn eines Knotens zu finden und die Entfernungen zu aktualisieren. Dies erfordert ein Durchlaufen der Kantenliste, was den Aufwand auf $O(E \log n)$ erhöht, da das Hauptaugenmerk auf der Effizienz der Prioritätswarteschlange für die Knotenauswahl liegt.
Adjazenzmatrix	$O(n^2)$	Bei der Adjazenzmatrix kann einfach die Zeile für jeden Knoten durchgegangen werden, um die Nachbarn zu finden und die Entfernungen zu aktualisieren. Dies erfordert $O(n)$ Operationen pro Knoten, und da der Algorithmus für alle Knoten ausgeführt wird, ergibt sich eine Laufzeit von $O(n^2)$.

2.2.3 Aufgabe: Einfluss auf den Aufwand von Floyd ✓

Begründen Sie inwiefern der Aufwand des Floyd Algorithmus sich für die beiden Datenstrukturen Adjazenzliste und Adjazenzmatrix unterscheidet. *Probeklausur Algorithmen der Informatik SS 2021 2021 Probeklausur Algorithmen der Informatik SoSe 2024 2024*

Datenstruktur	Bereich	Aufwand	Erklärung
Adjazenzmatrix	Laufzeit	$O(n^3)$	Der Zugriff auf die Kanteninformationen ist in konstanter Zeit $O(1)$, aber die dreifache Schleifenstruktur sorgt für die $O(n^3)$ Laufzeit insgesamt.
	Speicheraufwand	$O(n^2)$	Eine Adjazenzmatrix ist eine $n \times n$ Matrix
Adjazenzliste	Laufzeit	wc $O(n^3)$	Während die Adjazenzliste langsamer ist, weil wir durch die Liste der Nachbarn eines Knotens iterieren müssen, ist der grundlegende Algorithmus immer noch $O(n^3)$, da die Anzahl der Iterationen durch die Schleifenstruktur dominiert wird. Die Laufzeit kann im schlimmsten Fall in einem dichten Graphen ebenfalls $O(n^3)$ erreichen, da es für jede Kante in den Listen überprüft werden muss.
	Speicheraufwand	$O(n + E)$	eine Liste

2.2.4 Aufgabe: Einfluss auf asymptotischen Aufwand von Floyd ✓

Erklären Sie wie der asymptotische Aufwand des Floyd Algorithmus unter Verwendung der Datenstruktur Kantenliste und unter Verwendung der Adjazenzmatrix zustande kommt. Erklären Sie weiter, warum sich der Aufwand hier gleicht /unterscheidet! [Klausur Algorithmen der Informatik Sommersemester 2021 2021](#)

Der asymptotische Aufwand des Floyd-Warshall-Algorithmus bleibt $O(|V|^3)$, unabhängig davon, ob eine Adjazenzmatrix oder eine Kantenliste verwendet wird. Der Algorithmus führt immer drei geschachtelte Schleifen durch, die die Laufzeit dominieren. Die spezifische Datenstruktur beeinflusst die initiale Setup-Zeit, hat aber keinen Einfluss auf die asymptotische Komplexität der Hauptberechnungen des Algorithmus.

Der Speicheraufwand unterscheidet sich jedoch. Die Adjazenzmatrix benötigt $O(|V|^2)$ Speicher, während die Kantenliste $O(|E|)$ Speicher benötigt.

2.3 Eulerkreisalgorithmus

Eulerkreis

Ein Eulerkreis ist ein geschlossener Weg, der jede Kante des Graphen genau einmal durchläuft.
Voraussetzungen:

- zusammenhängender Graph
- jeder Knoten des Graphen hat einen geraden Grad

In der Vorlesung haben wir den **Hierholzer-Algorithmus** als Eulerkreisalgorithmus kennen gelernt.

Algorithm 1 Hierholzer-Algorithmus für einen Eulerkreis

Input Graph	Graph wird als Input eingegeben
if not G.connected() return False	Wenn nicht zusammenhängend, gibt es auch keinen Eulerkreis
end if	
for v = 0 to G.Nodes() if G.Degree(v) ist ungerade return False end if end for	alle Knoten durchgehen wenn ein Knoten eine ungerade Anzahl von Kanten hat dann gäbe es keinen Eulerkreis
<i>hGraph</i> ← Kopie von <i>G</i> <i>oTour</i> ≥ <i>ts</i> neuer Stack <i>hStack</i> ← neuer Stack <i>hStack.push</i> (<i>v</i> ₀)	um nicht originale Daten zu löschen um final Knoten des Kreises zu speichern um aktuelle Runde zu speichern Starte bei einem beliebigen Knoten <i>v</i> ₀
while <i>hStack.count</i> > 0 <i>v</i> ← <i>hStack.pop</i> () <i>oTour.push</i> (<i>v</i>) while <i>hGraph.degree</i> (<i>v</i>) > 0 <i>hStack.push</i> (<i>v</i>) <i>u</i> ← <i>hGraph.NextNode</i> (<i>v</i>) <i>hGraph.DeleteEdge</i> (<i>v</i> , <i>u</i>) <i>v</i> ← <i>u</i> end while end while	läuft, solange der Stack nicht leer ist fügt aktuellen Knoten in aktuelle Tour ein solange der aktuelle Knoten noch unbesuchte (nicht gelöschte) Kanten hat falls man später auf ihn zurückkehren muss, um andere Kanten zu durchlaufen damit Kante nicht erneut durchlaufen wird
return <i>oTour</i>	gibt gefundenen Eulerkreis zurück

2.3.1 Aufgabe: Einfluss auf den Aufwand vom Eulerkreisalgorithmus ✓

Gegeben ein Graph, für den ein Eulerkreis existiert. Erläutern Sie, wie sich der datenstrukturunabhängige Aufwand (wenn die Datenstruktur zur Repräsentation des Graphen noch nicht feststeht) für den Eulerkreisalgorithmus ergibt. Spezifizieren Sie außerdem, wie dieser für die Datenstruktur Kantenliste aussieht und erläutern Sie den Aufwand für die notwendige Operation. *Klausur Algorithmen der Informatik Sommersemester 2021 2021*

Datenstrukturunabhängiger Aufwand: $O(E)$

Jede Kante wird im schlimmsten Fall nur einmal durchlaufen. Daher ist der Aufwand proportional zur Anzahl der Kanten.

Aufwand für die Kantenliste: $O(E^2)$

Jedes Mal, wenn der Algorithmus eine Kante durchlaufen möchte, muss er sie in der Liste suchen. Das kostet im schlimmsten Fall $O(E)$ pro Kante.

2.3.2 Aufgabe: Eignung bezüglich Grapheneigenschaften ✓

Nehmen Sie an, Ihr Graph ist als Adjazenzliste gespeichert. Nennen Sie zwei verschiedene Speicherstrukturen für eine Priority Queue im Dijkstra Algorithmus. Nennen Sie für beide Speicherstrukturen jeweils einen Fall indem diese für den Algorithmus vorteilhaft ist! Erläutern Sie dies am Aufwand und erklären Sie, durch welche Operationen und mit welchem jeweiligen Aufwand sich dieser zusammensetzt. *Klausur Algorithmen der Informatik Sommersemester 2021 2021*

Datenstruktur	Aufwand		Vorteil
Ungeordnetes Array	Insert:	$O(1)$	Vorteilhaft bei kleinen Graphen, da das Einfügen von Knoten sehr effizient ist und die höheren Kosten der anderen Funktionen nicht so ins Gewicht fallen
	Extract-Min:	$O(n)$	
	Decrease-Key:	$O(n)$	
Binärer Min-Heap	Insert:	$O(\log n)$	vorteilhaft bei großen Graphen mit vielen Knoten, da die häufigen Operationen Extract-Min und Decrease-Key in logarithmischer Zeit durchgeführt werden, was bei großen Graphen zu einer erheblichen Zeitersparnis führt
	Extract-Min:	$O(\log n)$	
	Decrease-Key:	$O(\log n)$	

2.4 Queue

Unterschied zwischen Indexed Priority Queue und Priority Queue

	Priority Queue	Indexed Priority Queue
Operationen	Insert Find-Min Delete-Min	Insert Find-Min Delete-Min Decrease-Key/Increase-Key Change-Key Access via Index
Implementierungen	binäre Heaps Fibonacci-Heaps	binäre Heaps (+Indizes Array) Fibonacci-Heaps (+Indizes Array)
Zugriff	Zugriff nur über die Priorität (z.B. das Minimum)	Zugriff auf Elemente über einen Index und erlaubt das Ändern der Priorität eines bestimmten Elements
Verwendung	u.a. Heapsort	Dijkstra Prim

2.4.1 Aufgabe: Breitensuche ✓

Erklären Sie kurz warum die Queue für Breitensuche benutzt werden kann. [Probeklausur Algorithmen der Informatik SS 2021](#) [2021 Probeklausur Algorithmen der Informatik SoSe 2024](#) [2024](#)

Die Queue funktioniert nach dem First-In-First-Out (FIFO)-Prinzip, was bedeutet, dass der zuerst entdeckte Knoten auch zuerst verarbeitet wird. Dies entspricht genau dem Prinzip der Breitensuche, bei der die Knoten in der Reihenfolge ihres Entdeckens verarbeitet werden.

2.4.2 Aufgabe: Realisierung einer Priority Queue ✓

Nennen Sie zwei unterschiedliche Speicherstrukturen zur Realisierung einer Priority Queue. Wie aufwändig ist hier je das Finden eines Minimums / das Löschen eines Elements? Erläutern Sie kurz, wie sich die Laufzeit für das Löschen ergibt. *Probeklausur Algorithmen der Informatik SS 2021 2021 Probeklausur Algorithmen der Informatik SoSe 2024 2024*

Datenstruktur	Aufwand		Erklärung
Ungeordnetes Array	Element löschen:	$O(V)$	Das Löschen des Minimums erfordert zwei Schritte: • Suchen des Minimums $\rightarrow O(V)$ • Löschen des Minimums $\rightarrow O(1)$
	Minimum finden	$O(V)$	die gesamte Liste muss durchsucht werden, da die Elemente unsortiert sind
Binärer Min-Heap	Element löschen:	$O(\log V)$	Das Löschen des Minimums erfordert zwei Schritte: • Wurzelement entfernen • Heapify $\rightarrow O(\log V)$
	Minimum finden	$O(V)$	Minimum befindet sich immer an der Wurzel des Heaps, d.h., am ersten Element im Array
Fibonacci-Heap	Element löschen:	$O(\log V)$ (<i>amortisiert</i>)	Das Löschen des Minimums erfordert zwei Schritte: • Element m.H. der Hash-Tabelle oder des Arrays finden: $O(1)$ • Element entfernen durch markieren: $O(\log V)$
	Minimum finden	$O(1)$	Das Minimum befindet sich in einem speziellen Zeiger, der auf das kleinste Element in der Wurzel-Liste zeigt. Daher ist das Finden des Minimums ein konstanter Vorgang mit Aufwand

2.4.3 Aufgabe: Realisierung Indexed Priority Queue ✓

Nennen Sie zwei unterschiedliche Speicherstrukturen zur Realisierung einer Indexed Priority Queue. Wie aufwändig ist hier je das Finden eines Minimums / das Löschen eines Elements? Erläutern Sie kurz, wie sich die Laufzeit für das Löschen ergibt.

Amortisierter Aufwand

Amortisierte Analyse ist ein Konzept in der Informatik, das verwendet wird, um die durchschnittlichen Kosten einer Operation über eine Reihe von Operationen zu bestimmen, anstatt die Kosten einer einzelnen Operation isoliert zu betrachten. Diese Methode ist besonders nützlich, wenn einige Operationen in einer Datenstruktur teuer sind, während andere günstig sind. Die amortisierte Analyse hilft zu zeigen, dass trotz der teuren Einzeloperationen die durchschnittlichen Kosten pro Operation in einem längeren Zeitraum effizient bleiben.

Datenstruktur	Aufwand	Erklärung
Binärer Heap mit Hash-Tabelle	Element löschen:	$O(\log V)$ Das Löschen des Minimums erfordert zwei Schritte: • Position im Heap finden: $O(1)$ • Heap-Eigenschaft wieder herstellen: $O(\log V)$ da der binäre Heap eine Höhe von höchstens $\log V$ hat
	Minimum finden	$O(1)$ Das Minimum ist immer an der Wurzel des Heaps (Index 0 im Array). Daher hat das Finden des Minimums einen konstanten Aufwand von $O(1)$
Fibonacci-Heap	Element löschen:	$O(\log V)$ (amortisiert) Das Löschen des Minimums erfordert zwei Schritte: • Element m.H. der Hash-Tabelle oder des Arrays finden: $O(1)$ • Element entfernen durch markieren: $O(\log V)$
	Minimum finden	$O(1)$ Das Minimum befindet sich in einem speziellen Zeiger, der auf das kleinste Element in der Wurzel-Liste zeigt. Daher ist das Finden des Minimums ein konstanter Vorgang mit Aufwand

2.4.4 Einfluss der Realisierung auf den Aufwand von Dijkstra ✓

Nennen Sie für den Dijkstra-Algorithmus und zwei verschiedene Realisierungen der Priority Queue den entstehenden Aufwand unter der Annahme, dass der Graph als Adjazenzliste gespeichert wird. Erläutern Sie dabei, wie sich der Aufwand zusammensetzt (d.h. welche Operationen bestimmen das jeweilige Ergebnis und was ist deren Aufwand?).
Probeklausur Algorithmen der Informatik SS 2021 2021 Probeklausur Algorithmen der Informatik SoSe 2024 2024

	Dijkstra mit Binärer Heap	Dijkstra mit Fibonacci-Heap
Insert	$O(V \log V)$	$O(V)$
Erklärung	$ V $ Einfügeoperationen à $O(\log V)$	$ V $ Einfügeoperationen à $O(1)$
Extract-Min	$O(V \log V)$	$O(V \log V)$
Erklärung	$ V $ extract-min Operationen à $O(\log V)$	$ V $ extract-min Operationen à $O(\log V)$
Decrease-Key	$O(E \log V)$	
Erklärung	$ E $ decrease Operationen à $O(\log V)$	$ E $ decrease Operationen à $O(E)$
Gesamtlaufzeit	Insert + Extract-Min + Decrease-Key	
	$O(V \log V) + O(V \log V) + O(E \log V)$	$O(V) + O(V \log V) + O(E)$
	$= O(V \log V) + O(E \log V)$	
	$= O(\max(V , E) \log V)$	
	$O((V + E) \log V)$	$O(V \log V + E)$

3 Algorithmen

3.0.1 Aufgabe: Algorithmus für negative Kantengewichte ✓

Geben Sie eine konkrete Anwendung an, in der relevant ist, dass gewichtete Graphen auch negative Kantengewichte haben können. Nennen Sie einen Algorithmus, der für negative Kantengewichte geeignet ist. *Probeklausur Algorithmen der Informatik SS 2021 2021 Probeklausur Algorithmen der Informatik SoSe 2024 2024*

→ Währungsarbitrage

Und für negative Kantengewichte nutzt man den Bellman-Ford-Algorithmus.

3.1 Dijkstra

Dijkstra Algorithmus

Dijkstra findet die kürzesten Pfade in Graphen mit positiven Gewichten.

Er verwendet eine Priority Queue, um die Knoten mit der kürzesten bekannten Distanz effizient zu verarbeiten. Der Algorithmus funktioniert nur mit nicht-negativen Gewichten, da er sonst falsche Ergebnisse liefern würde.

3.1.1 Aufgabe: Eigenschaft zum Zeigen der Korrektheit ✓

Welche Eigenschaft, die elementar zum Zeigen der Korrektheit ist, behält der erstellte kürzeste Wegebau beim Dijkstra-Algorithmus in jedem Iterationsschritt? *Probeklausur Algorithmen der Informatik SS 2021 2021 Probeklausur Algorithmen der Informatik SoSe 2024 2024*

Die **Optimale-Substruktur-Eigenschaft**

3.2 Lineare Probleme

Unterschied zwischen LP und ILP

	IP	ILP
Variablen	beliebige Werte (auch reelle Zahlen)	ganzzahlige Zahlen
Komplexität der Lösung	relativ einfach	NP-schwer
Anwendungsbereiche	Probleme mit kontinuierlichen Entscheidungen z.B. bei der Optimierung von Produktionsmengen oder Transportwegen	Probleme mit Entscheidungen wie „ja/nein“ oder „ganzzahlige Mengen“ z.B. bei der Zuteilung von Ressourcen, bei Routing-Problemen oder bei Zeitplanungen
Lösungsmethode	direkte mathematische Methoden	ombinatorische Optimierungsansätze
Beispiel	Maximiere $z = 2x_1 + 3x_2$ unter den Nebenbedingungen: $x_1 + x_2 \leq 10$ $x_1, x_2 \geq 0$	Maximiere $z = 2x_1 + 3x_2$ unter den Nebenbedingungen: $x_1 + x_2 \leq 10$ $x_1, x_2 \geq 0$ $x_1, x_2 \in \mathbb{Z}$

3.2.1 Aufgabe: Relaxation eines ILP

→ Die Relaxation eines ILP vergrößert den Lösungsraum, da die Ganzzahligkeitsbedingungen wegfallen!

Was ist die Idee hinter einer sogenannten Relaxation eines ILP? [Probeklausur Algorithmen der Informatik SS 2021](#) [2021 Probeklausur Algorithmen der Informatik SoSe 2024](#) [2024](#)

Die Idee ist, die Schwierigkeit des Problems zu reduzieren, indem die Ganzzahligkeitsbedingungen für die Variablen aufgehoben werden. Dadurch wird das Problem in ein LP umgewandelt, das einfacher und schneller zu lösen ist.

3.2.2 Aufgabe: Lösung einer Relation ✓

Nehmen Sie an, ein ILP (A) sei gegeben mit einer zugehörigen Relaxation (B). Gibt es eine Relation zwischen der Menge der zulässigen Lösungen für (A) und (B)? [Probeklausur Algorithmen der Informatik SS 2021 2021](#)
[Probeklausur Algorithmen der Informatik SoSe 2024 2024](#)

Ja, es gibt eine klare Relation zwischen der Menge der zulässigen Lösungen für das ILP (A) und seiner Relaxation (B).

Die Menge der zulässigen Lösungen des ILP (A) ist eine Teilmenge der Menge der zulässigen Lösungen der Relaxation (B):

$$\text{Zulässige Lösungen von (A)} \subseteq \text{Zulässige Lösungen von (B)}$$

Denn die Lösungsmenge des LP ist nicht durch die Anforderung der Ganzzahligkeit beschränkt.

3.2.3 Aufgabe: Relation zwischen optimalen Werten ✓

Nehmen Sie an, ein ILP (A) sei gegeben mit einer zugehörigen Relaxation (B). Gibt es eine Relation zwischen dem optimalen Wert von (A) und (B)? *Probeklausur Algorithmen der Informatik SS 2021 2021 Probeklausur Algorithmen der Informatik SoSe 2024 2024*

Maximierungsproblem

$$\text{optimaler Wert der Relaxation (B)} \geq \text{optimaler Wert des ILP (A)}$$

Der optimale Wert der Relaxation (B) ist größer oder gleich dem optimalen Wert des ILP (A), da die Relaxation mehr zulässige Lösungen hat

Minimierungsproblem

$$\text{optimaler Wert der Relaxation (B)} \leq \text{optimaler Wert des ILP (A)}$$

Der optimale Wert der Relaxation (B) ist kleiner oder gleich dem optimalen Wert des ILP (A), aus demselben Grund.

Satisfiability Problem

Das SAT-Problem ist eines der grundlegendsten Entscheidungsprobleme in der Informatik. Es fragt, ob eine boolesche Formel erfüllbar ist, d. h., ob es eine Belegung der Variablen gibt, die die Formel wahr macht.

Maximum Satisfiability Problem

MAXSAT ist eine Erweiterung des SAT-Problems, bei dem es darum geht, die maximale Anzahl von erfüllbaren Klauseln in einer booleschen Formel zu finden.

Umwandlung eines MAXSAT-Problems in ein LIP

1. Formuliere das MAXSAT-Problem:

Was wird zu was?

2. Definieren der Variablen

3. Einführung der binären Variablen

4. Umwandlung der Klauseln zu Ungleichungen

X wird zu... $x \geq 1$

$X1 \vee X2$ wird zu... $x_1 + x_2 \geq 1$

$\neg X$ wird zu... $(1 - x) \geq 1$

$\neg X1 \vee X3$ wird zu... $(1 - x_1) + x_3 \geq 1$

3.2.4 MAXSAT zu ILP I. ✓

Das MAXSAT Problem ist das Problem, Boolesche Variablen in einer logischen Formel in konjunktiver Normalform so zu belegen, dass möglichst viele Klauseln erfüllt sind. Wandeln Sie folgendes MAXSAT in ein ILP um: [Probeklausur Algorithmen der Informatik SS 2021](#) [2021 Probeklausur Algorithmen der Informatik SoSe 2024](#) [2024](#)

1. Formulierung

$$(X1 \vee X2) \wedge (\neg X3 \vee \neg X2 \vee X1) \wedge (\neg X1 \vee X3)$$

2. Definierung der Variablen

$$x_1, x_2, x_3$$

3. binäre Variablen einführen

$$x_i = 1 \text{ (wahr)}$$

$$x_i = 0 \text{ (falsch)}$$

(ebenso für z_i)

4. Umwandlung

$$(X1 \vee X2)$$

$$x_1 + x_2 \geq 1$$

$$(\neg X3 \vee \neg X2 \vee X1)$$

$$(1 - x_3) + (1 - x_2) + x_1 \geq 1$$

bzw. vereinfacht

$$x_1 - x_2 - x_3 \geq -1$$

$$(\neg X1 \vee X3)$$

$$(1 - x_1) + x_3 \geq 1$$

bzw. vereinfacht

$$-x_1 + x_3 \geq 0$$

Zusammenfassung

Maximiere $z_1 + z_2 + z_3$ *

$$z_1 : x_1 + x_2 \geq 1$$

$$z_2 : x_1 - x_2 - x_3 \geq -1$$

$$z_3 : -x_1 + x_3 \geq 0$$

* $z_1 + z_2 + z_3$ zu maximieren bedeutet möglichst viele der Formeln zu erfüllen.

3.2.5 MAXSAT zu ILP II. ✓

Das MAXSAT Problem ist das Problem, Boolesche Variablen in einer logischen Formel in konjunktiver Normalform so zu belegen, dass möglichst viele Klauseln erfüllt sind. Wandeln Sie folgendes MAXSAT in ein ILP um: [Klausur Algorithmen der Informatik Sommersemester 2021 2021](#)

1. Formulierung

$$(\neg X1 \vee X2) \wedge (X2 \vee X3) \wedge (\neg X1 \vee \neg X4)$$

2. Definierung der Variablen

$$x_1, x_2, x_3, x_4$$

3. binäre Variablen einführen

$$x_i = 1 \text{ (wahr)}$$

$$x_i = 0 \text{ (falsch)}$$

(ebenso für z_i)

4. Umwandlung

$$(\neg X1 \vee X2)$$

$$(1 - x_1) + x_2 \geq 1$$

$$(X2 \vee X3)$$

$$x_2 + x_3 \geq 1$$

$$(\neg X1 \vee \neg X4)$$

$$(1 - x_1) + (1 - x_4) \geq 1$$

Zusammenfassung

$$\text{Maximiere } z_1 + z_2 + z_3^*$$

$$z_1 : (1 - x_1) + x_2 \geq 1$$

$$z_2 : x_2 + x_3 \geq 1$$

$$z_3 : (1 - x_1) + (1 - x_4) \geq 1$$

* $z_1 + z_2 + z_3$ zu maximieren bedeutet möglichst viele der Formeln zu erfüllen.

3.3 Floyd-Warshall

Floyd-Warshall Algorithmus

Der Algorithmus arbeitet in drei verschachtelten Schleifen.

Äußerste Schleife (über k): Diese Schleife durchläuft alle Knoten k und stellt sicher, dass Knoten k als möglicher Zwischenknoten verwendet werden kann. Nach jedem Durchlauf ist garantiert, dass die kürzesten Wege zwischen allen Knotenpaaren unter Berücksichtigung der Knoten $1, 2, \dots, k$ als Zwischenknoten gefunden werden.

Mittlere Schleife (über i): Diese Schleife iteriert über die Startknoten i der Pfade. Sie prüft alle möglichen Startpunkte i , von denen aus kürzeste Wege zu anderen Knoten j berechnet werden.

Innerste Schleife (über j): Diese Schleife iteriert über die Zielknoten j der Pfade. Für jedes Knotenpaar (i, j) prüft der Algorithmus, ob der aktuelle kürzeste bekannte Weg von i nach j durch den Knoten k verkürzt werden kann.

3.3.1 Eigenschaft der Distanzmatrix ✓

Beschreiben Sie in eigenen Worten, welche Eigenschaft die Distanzmatrix im Floyd-Warshall Algorithmus nach jedem Iterationsschritt der äußersten (also ersten) Schleife, in Bezug auf gefundene kürzeste Wege, erfüllt. [Klausur Algorithmen der Informatik Sommersemester 2021 2021](#)

Nach jedem Iterationsschritt der äußersten Schleife des Floyd-Warshall-Algorithmus enthält die Distanzmatrix die Längen der kürzesten Wege, die nur Knoten bis zum aktuellen Iterationsknoten als Zwischenknoten verwenden.

3.4 Weighted Vertex Cover Problem ✓

Ein **Vertex Cover** ist eine Menge von Knoten in einem Graphen, die alle Kanten abdeckt. Das heißt, jede Kante im Graphen ist mit mindestens einem Knoten aus dieser Menge verbunden.

Ein **Weighted Vertex Cover** ist eine erweiterte Version des Vertex Cover Problems, bei dem jedem Knoten im Graphen ein Gewicht zugeordnet ist.

Wichtig!: das Weighted Vertex Cover Problem ist NP-hart.

Algorithm 3 Weighted Vertex Cover

$C := \emptyset$	leere Menge, um die ausgewählten Knoten zu speichern
$E' := E$	Kopie der Kantenmenge, die im Laufe des Algorithmus verkleinert wird
while $E' \neq \emptyset$ do	
wähle $(u, v) \in E'$	beliebige Kante auswählen
$C := C \cup \{u, v\}$	aktuelle Knoten werden zur Ergebnisliste zugefügt
$E' := E' \setminus \{(u', v') \mid u' \in \{u, v\} \vee v' \in \{u, v\}\}$	alle von den aktuellen Knoten abgehenden Kanten werden aus der Kantenmenge gelöscht
end while	

3.4.1 Aufgabe: Algorithmus zur Approximation ✓

Betrachten Sie den Algorithmus zur Approximation des Weighted Vertex Cover Problems aus der Vorlesung. Erklären Sie warum der letzte Schritt, das Runden der LP Lösung, wieder ein gültiges Vertex Cover ergibt. [Klausur Algorithmen der Informatik Sommersemester 2021 2021](#)

Der letzte Schritt, das Runden der LP-Lösung, garantiert, dass jede Kante des Graphen weiterhin abgedeckt bleibt. Da jede Kante bereits in der LP-Lösung durch mindestens einen Knoten mit einem Wert von mindestens $\frac{1}{2}$ abgedeckt ist, wird nach dem Runden der entsprechenden Knoten in das Vertex Cover aufgenommen, wodurch ein gültiges Vertex Cover entsteht.

3.5 Kruskal

3.5.1 Programmierparadigma von Kruskal

In der Vorlesung wurden drei verschiedene Programmierparadigmen behandelt. Auf welchem von diesen basiert der Algorithmus von Kruskal? Beschreiben Sie die Grundidee dieses Programmierparadigmas in Worten und wie diese im besagten Algorithmus Verwendung findet. Beschreiben Sie auch die Grundidee der anderen beiden Programmierparadigmen und erläutern Sie, warum der Algorithmus nicht auf diesen basiert! *Klausur Algorithmen der Informatik Sommersemester 2021 2021*

→ Greedy

Das greedy Paradigma basiert auf der Idee, bei jedem Schritt eine lokal optimale Entscheidung zu treffen, in der Hoffnung, dass dies zu einer global optimalen Lösung führt. Greedy-Algorithmen wählen in jedem Schritt das Element aus, das augenblicklich den größten Nutzen oder die geringsten Kosten verspricht, ohne Rücksicht auf spätere Entscheidungen.

Der Kruskal-Algorithmus folgt diesem Paradigma, indem er in jedem Schritt die jeweils leichteste Kante auswählt, die zwei bisher unverbundene Komponenten des Graphen verbindet. Durch das Hinzufügen der Kante wird der Gesamtbaum schrittweise aufgebaut, bis alle Knoten verbunden sind.

Dynamische Programmierung:

Probleme lösen, indem sie in Teilprobleme zerlegt werden. Diese Teilprobleme werden jeweils nur einmal gelöst und ihre Lösungen werden gespeichert (memoization), um die Lösung des Gesamtproblems effizient zu berechnen.

→ Der Kruskal-Algorithmus benötigt keine wiederholte Lösung von Teilproblemen.

Iterative Verbesserung:

Bei der Iterativen Verbesserung wird ausgehend von einer anfänglichen Lösung sukzessive versucht, durch kleine Anpassungen oder lokale Veränderungen eine bessere Lösung zu finden. Dies wird solange wiederholt, bis eine optimale oder zumindest verbesserte Lösung gefunden wird oder keine weiteren Verbesserungen mehr vorgenommen werden können.

→ Kruskal hat keine schrittweise Optimierung und verbessert generell keine bestehenden Lösungen.

4 Anwendung

4.1 Graph 1

Gegeben ist der gerichtete, gewichtete Graph $G = (V, (E, w))$ mit $V = 1, 2, 3, 4, 5, 6$ und den gewichteten Kanten $1 \xrightarrow{8} 2, 1 \xrightarrow{7} 3, 2 \xrightarrow{18} 3, 2 \xrightarrow{7} 4, 3 \xrightarrow{4} 5, 4 \xrightarrow{10} 3, 4 \xrightarrow{11} 6, 5 \xrightarrow{9} 6$ *Probeklausur Algorithmen der Informatik SS 2021 2021*

4.1.1 Finde Eulerpfad bzw. Eulerkreis ✓

Nehmen Sie an, die Kanten seien ungerichtet und ungewichtet. Gibt es einen Eulerpfad, gibt es einen Eulerkreis? Warum? Falls ja, geben Sie diesen an! *Probeklausur Algorithmen der Informatik SS 2021 2021*

Die Knoten 2 und 3 haben einen ungeraden Grad, daher gibt es keinen Eulerkreis. Aber da es nur zwei Knoten mit ungeradem Grad gibt, gibt es einen Eulerpfad.

4.1.2 Änderung der Distanzmatrix ✓

Nehmen Sie an, die Kanten sind ungerichtet. Welche Einträge der Distanzmatrix ändern sich nach dem zweiten Durchlauf der äußersten Schleife des Floyd Algorithmus? Geben Sie auch an welchen Wert diese annehmen. Initiale Distanzmatrix D : *Probeklausur Algorithmen der Informatik SS 2021 2021*

$$\begin{pmatrix} 0 & 8 & 7 & \infty & \infty & \infty \\ 8 & 0 & 18 & 7 & \infty & \infty \\ 7 & 18 & 0 & 10 & 4 & \infty \\ \infty & 7 & 10 & 0 & \infty & 11 \\ \infty & \infty & 4 & \infty & 0 & 9 \\ \infty & \infty & \infty & 11 & 9 & 0 \end{pmatrix}$$

Geänderte Distanzmatrix:

$$\begin{pmatrix} 0 & 8 & 7 & 15 & \infty & \infty \\ 8 & 0 & 18 & 7 & \infty & \infty \\ 7 & 18 & 0 & 10 & 4 & \infty \\ 15 & 7 & 10 & 0 & \infty & 11 \\ \infty & \infty & 4 & \infty & 0 & 9 \\ \infty & \infty & \infty & 11 & 9 & 0 \end{pmatrix}$$

Begründung:

In der zweiten Iteration werden die Knoten 2 ($d[i][2]$ bzw. $d[2][j]$) als Zwischenknoten betrachtet.

$d[1][4]$:

Der direkte Weg von Knoten 1 zu Knoten 4 ist unendlich (∞).

Der Weg über Knoten 2: $d[1][2] + d[2][4] = 8 + 7 = 15$.

Da $15 < \infty$, wird $d[1][4]$ auf 15 gesetzt.

$d[4][1]$:

Der direkte Weg von Knoten 4 zu Knoten 1 ist unendlich (∞).

Der Weg über Knoten 2: $d[4][2] + d[2][1] = 7 + 8 = 15$.

Da $15 < \infty$, wird $d[4][1]$ auf 15 gesetzt.

Für $d[3][2]$ und $d[2][3]$ ändert sich bei einem Weg über Knoten 2 nichts.

4.1.3 Der Graph als Flussnetzwerk

Nehmen Sie an, der Graph sei ein Flussnetzwerk mit Source 1 und Target 6 mit gerichteten Kanten. Geben Sie einen augmentierenden Pfad gemäß des Edmond-Karp Algorithmus an, wenn man mit dem Fluss 0 startet. Geben Sie auch die residuale Kapazität und das entsprechende residuale Netzwerk nach der Augmentierung mit diesem Pfad an. Notieren Sie hierzu, welche Kanten hinzukommen/verschwinden und wie sich die Kapazitäten der Kanten verändern. *Probeklausur Algorithmen der Informatik SS 2021 2021*

Augmentierender Pfad

Per Breitensuche einen Pfad von Quelle zu Senke finden, der keine gesperrten Kanten enthält.

Möglichkeit 1.

Pfad $1 \rightarrow 3 \rightarrow 5 \rightarrow 6$

Möglichkeit 2.

Pfad $1 \rightarrow 2 \rightarrow 4 \rightarrow 6$

Residualkapazitäten ermitteln

Basierend auf der im Pfad vorhandenen kleinsten Kapazität, um die der Fluss entlang der Pfade dann erhöht wird.

Möglichkeit 1.

Kleinste Kapazität: 4 (von $3 \xrightarrow{4} 5$)

$1 \xrightarrow{4/7} 3 \xrightarrow{4/4} 5 \xrightarrow{4/9} 6$

Residualkapazitäten:

Kante	Kapazität
$1 \rightarrow 2$	8
$1 \rightarrow 3$	3
$2 \rightarrow 3$	18
$2 \rightarrow 4$	7
$3 \rightarrow 5$	0
$4 \rightarrow 3$	10
$4 \rightarrow 6$	11
$5 \rightarrow 6$	5

Möglichkeit 2.

Kleinste Kapazität: 7 (von $2 \xrightarrow{7} 4$)

$1 \xrightarrow{7/8} 2 \xrightarrow{7/7} 4 \xrightarrow{7/11} 6$

Residualkapazitäten:

Kante	Kapazität
$1 \rightarrow 2$	1
$1 \rightarrow 3$	7
$2 \rightarrow 3$	18
$2 \rightarrow 4$	0
$3 \rightarrow 5$	4
$4 \rightarrow 3$	10
$4 \rightarrow 6$	4
$5 \rightarrow 6$	9

Fluss augmentieren

Den Fluss mit den neuen Kapazitäten updaten.

4.1.4 Optimaler Fluss

Geben Sie einen optimalen Fluss an und argumentieren Sie mit Hilfe des max-flow-min-cut Theorems, dass Ihr Fluss optimal ist. [Probeklausur Algorithmen der Informatik SS 2021 2021](#)

Laut des max-flow-min-cut Theorems sind bei einem optimalen Fluss der Wert des maximalen Flusses und des minimalen Schnittes gleich.

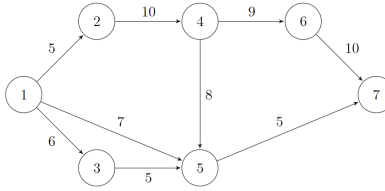
Man führt die beiden Augmentierungsschritte durch, die oben angegeben sind und bekommt daraufhin ein Flussnetzwerk mit dem maximalen Fluss $4 + 7 = 11$. Der minimale Schnitt ist hier mit dem Gewicht der Kanten $2 \rightarrow 4$ und $3 \rightarrow 5$ ebenfalls $4 + 7 = 11$.

Minimum Cut

Das kleinste Gesamtgewicht der Kanten, die, wenn man sie entfernen würde, die Quelle von der Senke trennen würden. Wikipedia contributors [2024](#)

4.2 Graph 2

Klausur Algorithmen der Informatik Sommersemester 2021 2021



4.2.1 Finde Eulerpfad bzw. Eulerkreis

Nehmen Sie an, die Kanten seien ungerichtet und ungewichtet. Gibt es einen Eulerpfad bzw. einen Eulerkreis? Warum/Warum nicht? Falls ja, geben Sie diesen an! *Klausur Algorithmen der Informatik Sommersemester 2021 2021*

4.2.2 Ergebnis von Kruskal

Nehmen Sie nun an, der Graph sei ungerichtet und gewichtet. Was ist das Ergebnis des Algorithmus von Kruskal, wenn dieser nach der 5-ten Iteration gestoppt wird? Geben Sie die gefundenen Kanten zu diesem Zeitpunkt an, in der Reihenfolge, wie sie vom Algorithmus ausgewählt werden! *Klausur Algorithmen der Informatik Sommersemester 2021 2021*

4.2.3 Kürzeste Pfade Baum

Nehmen Sie an, dass die Kanten gerichtet und gewichtet sind. Wie sieht der durch den Dijkstra-Algorithmus gefundene kürzeste Pfadbaum mit Start in 1 aus, wenn der Algorithmus nach 5 Iterationsschritten der äußersten Schleife gestoppt wird. Geben Sie den kürzeste Pfadbaum, wie in der Vorlesung, durch die beiden Arrays *pred* (Vorgänger) und *pi* (kürzeste Wege) an. *Klausur Algorithmen der Informatik Sommersemester 2021 2021*

4.2.4 Der Graph als Flussnetzwerk

Nehmen Sie an, der Graph sei ein Flussnetzwerk mit Source 1 und Target 7 mit gerichteten Kanten. Geben Sie einen maximalen Fluss für diesen Graphen an, welcher mit dem Edmond-Karp Algorithmus gefunden werden kann. Zeigen Sie mit einem aus der Vorlesung bekannten Verfahren, dass dieser Fluss maximal ist. *Klausur Algorithmen der Informatik Sommersemester 2021 2021*

5 Linear Programming

5.1 Verwandeln Sie folgendes LP in ein LP in Standardform

Ein LP in Standardform bringen

Ein LP in Standardform hat folgende Eigenschaften:

- nur nicht-negative Variablen
- Gleichungen anstatt Ungleichungen

Also müssen die Ungleichungen in Gleichungen umgewandelt werden, negative Variablen in positive umgewandelt werden und anschließend wird die Zielfunktion umformuliert.

a.

$$\begin{aligned} \min \quad & x_1 + 2x_2 - x_3 \\ \text{so dass} \quad & x_2 - x_3 = 23 \\ & x_1 \geq 0, \quad x_3 \leq 0 \end{aligned}$$

Figure 1: *Probeklausur Algorithmen der Informatik SoSe 2024 2024*

b.

$$\begin{aligned} \min \quad & -2x_1 - x_2 + 2.5x_3 \\ \text{so dass} \quad & x_1 - x_3 = 94 \\ & x_1 \leq 0, \quad x_2 \geq 0 \end{aligned}$$

Figure 2: *Klausur Algorithmen der Informatik Sommersemester 2021 2021*

c. *Probeklausur Algorithmen der Informatik SS 2021 2021*

$$\begin{aligned} \min \quad & 2x_1 + 2x_2 \\ \text{so dass} \quad & x_1 + x_2 \leq 42 \\ & x_1 \leq 0 \end{aligned}$$

Schlupfvariablen

Schlupfvariablen sind spezielle Variablen, die eingeführt werden, um Ungleichungen in Gleichungen zu verwandeln.

Zum Beispiel: Wenn eine Nebenbedingung

$$x_1 + 2x_2 \leq 4$$

lautet, wird eine Schlupfvariable $s_1 \geq 0$ hinzugefügt, um dies in eine Gleichung zu verwandeln:

$$x_1 + 2x_2 + s_1 = 4$$

Negative Variablen in positive und Ungleichungen in Gleichungen umwandeln m.H. von Schlupfvariablen

$$x_1 \leq 0$$

wird zu $x_1 = -z_1$

$$\text{mit } z_1 \geq 0$$

und

$$x_1 + x_2 \leq 42$$

wird zu $x_1 + x_2 + z_2 = 42$

$$\text{mit } z_2 \geq 0$$

Daraus kann man direkt formen:

$$\begin{aligned} & -z_2 + x_2 + z_1 = 42 \\ \text{mit } & x_2, z_1, z_2 \geq 0 \end{aligned}$$

Zielfunktion umformulieren

$$\begin{aligned} \text{aus } & \text{minimiere } 2x_1 + 2x_2 \\ \text{wird } & \text{minimiere } 2(-z_2) + 2x_2 \\ \text{bzw. } & \text{minimiere } -2z_2 + 2x_2 \end{aligned}$$

Daraus entsteht:

$$\begin{aligned} & \text{minimiere } 2x_1 + 2x_2 \\ & \text{so dass } -z_2 + x_2 + z_1 = 42 \\ & \text{mit } x_2, z_1, z_2 \geq 0 \end{aligned}$$

Basisvariablen

Basisvariablen sind die Variablen, die in einer Basislösung eines linearen Programms nicht auf Null gesetzt werden.

Um sie zu finden, muss die Gleichung zuerst in Standard- oder Schlupfform gebracht werden.

In der Ausgangsform des Simplex-Verfahrens wählt man typischerweise die Schlupfvariablen als Basisvariablen.

Umwandlung in Schlupfform

Prinzip:

$$x_1 + x_2 \leq n$$

wird zu

$$x_3 = n - x_1 - x_2$$

Beispiel: Schulz 2024

$$15x_1 + 5x_2 \leq 480$$

wird zu

$$x_3 = 480 - 15x_1 - 5x_2$$

In einem linearen Programm hat eine Basislösung in jeder Zeile des Gleichungssystems genau eine Basisvariable. Dies ist eine direkte Folge der mathematischen Struktur des Simplex-Verfahrens und der Definition von Basisvariablen.

Das **muss** so sein, damit man eine eindeutige Basislösung hat.

5.2 Gegeben sei folgendes LP in Schlupf-Form . Was ist die Basislösung? Ist diese zulässig? Warum/Warum nicht?

Bestimmung der Basislösung eines LPs und deren Zulässigkeit

Um eine Basislösung zu bestimmen, setzt man zuerst die Nicht-Basisvariablen (frei wählbare Variablen) auf 0 und berechnet dann die Werte der abhängigen Variablen.
Eine Basislösung ist zulässig, wenn alle Variablen nicht negativ sind.
Eine Basislösung ist optimal, wenn sie den höchsten möglichen Wert der Zielfunktion erreicht.

Basislösung

Man muss sowohl die Werte der Basisvariablen, als auch den Wert der Zielfunktion z berechnen!
Ansonsten ist die Basislösung nicht vollständig.

a.

$$\begin{aligned}\max z &= 8x_1 - x_2 \\ \text{so dass } x_3 &= 0.5x_1 - x_2 \\ x_4 &= -x_1 + 2x_2 \\ x_1, x_2, x_3, x_4 &\geq 0\end{aligned}$$

Figure 3: *Probeklausur Algorithmen der Informatik SoSe 2024 2024*

b.

$$\begin{aligned}\max z &= 6x_1 - 3x_2 \\ \text{so dass } x_3 &= x_1 - x_2 \\ x_4 &= -x_1 + 2x_2 + 1 \\ x_1, x_2, x_3, x_4 &\geq 0\end{aligned}$$

Figure 4: *Klausur Algorithmen der Informatik Sommersemester 2021 2021*

c.

Was ist die Basislösung in folgendem LP? Ist diese Basislösung zulässig? Welchen Wert hat die Zielfunktion für die Basislösung? Ist die Basislösung optimal? Warum / warum nicht? *Probeklausur Algorithmen der Informatik SS 2021 2021*

$$\begin{aligned}\text{maximiere } z &= x_1 - x_2 \\ \text{so dass } x_3 &= x_1 + x_2 + 3 \\ x_4 &= 2x_1 - x_2 + 2 \\ x_1, x_2, x_3, x_4 &\geq 0\end{aligned}$$

Basislösung

$$\begin{aligned}x_3 &= 0 + 0 + 3 = 3 \\ x_4 &= 2 * 0 - 0 + 2 = 2\end{aligned}$$

$$(x_1, x_2, x_3, x_4) = (0, 0, 3, 2)$$

Zulässigkeit

Alle Variablen sind Positiv, daher ist die Basislösung zulässig.

Wert für die Zielfunktion und Optimalität

$$z = x_1 + x_2 = 0 - 0 = 0$$

Würde man ein größeres x_1 wählen, so wäre auch der Wert der Zielfunktion größer. Daher ist die Lösung nicht optimal.

Standardform vs. Schlupfform

Formale Definition der Standardform:

$$\begin{aligned} \max \quad & \sum_{j=1}^n c_j x_j \\ \text{so dass} \quad & \sum_{j=1}^n a_{ij} x_j \leq b_i \quad \text{für alle } 1 \leq i \leq m \\ & x_j \geq 0 \quad \text{für alle } 1 \leq j \leq n \end{aligned}$$

- Ungleichungen **nicht erlaubt**
- Variablen **nicht negativ**
- die Nicht-Basisvariablen und Basisvariablen stehen "**frei**" in den Gleichungen herum

Beispiel:

$$\begin{aligned} \max \quad & z = 3x_1 + 2x_2 \\ \text{so dass} \quad & x_1 + 2x_2 + s_1 = 4 \\ & 3x_1 + 2x_2 - s_2 = 6 \\ \text{und} \quad & x_1, x_2, s_1, s_2 \geq 0 \end{aligned}$$

Formale Definition der Schlupfform:

$$\begin{aligned} \max \quad & z = \nu + \sum_{j \in N} c_j x_j \\ \text{so dass} \quad & x_i = b_i - \sum_{j \in N} a_{ij} x_j \quad \text{für } i \in B \\ & x_i \geq 0 \quad \text{für } i \in B \cup N \end{aligned}$$

- Ungleichungen **nicht erlaubt**
- Variablen **nicht negativ**
- Basisvariablen **in Abhängigkeit** von Nicht-Basisvariablen dargestellt

Beispiel:

$$\begin{aligned} \max \quad & z = 3x_1 + 2x_2 \\ \text{so dass} \quad & s_1 = 4 - x_1 - 2x_2 \\ & s_2 = 3x_1 + 2x_2 - 6 \\ \text{und} \quad & x_1, x_2, s_1, s_2 \geq 0 \end{aligned}$$

5.3 Simplex-Verfahren

Algorithm 4 Simplex-Verfahren

```
finde Initiallösungen
while Lösung ist nicht optimal do
    Pivotschritt zu einem vielversprechenden nächsten Extremwert
end while
```

5.3.1 Führen Sie in folgendem LP einen Pivotschritt durch:

Entering und Leaving Variables

Beispielrechnung

$$\begin{aligned}\max \quad & z = 23x_1 + 13x_2 \\ \text{mit} \quad & x_3 = 480 - 15x_1 - 5x_2 \\ & x_4 = 160 - 4x_1 - 4x_2 \\ & x_5 = 1190 - 20x_1 - 35x_2 \\ & x_1, x_2, x_3, x_4, x_5 \geq 0\end{aligned}$$

Die Entering-Variable könnte x_1 oder x_2 sein, da:

$$z = 23x_1 + 13x_2$$

Beide würden die Lösung verbessern. Wir wählen einfach mal x_1 .

Jetzt müssen wir berechnen was der Wert (die Ratio) von dem Koeffizienten der Entering-Variable durch das jeweilige "b" der Gleichung ist.

$$\begin{aligned}\max \quad & z = 23x_1 + 13x_2 \\ \text{mit} \quad & x_3 = 480 - 15x_1 - 5x_2 \\ & x_4 = 160 - 4x_1 - 4x_2 \\ & x_5 = 1190 - 20x_1 - 35x_2 \\ & x_1, x_2, x_3, x_4, x_5 \geq 0\end{aligned}$$

Die Ratios ergeben:

$$\begin{aligned}\frac{480}{15} &= 32 && \text{für } x_3 \\ \frac{160}{4} &= 40 && \text{für } x_4 \\ \frac{1190}{20} &= 59.2 && \text{für } x_5\end{aligned}$$

Die Leaving-Variable wird die mit der kleinsten Ratio, also x_3 .

Nun formen wir die zur Leaving-Variable (x_3) gehörende Gleichung nach der Entering-Variable (x_1) um.

$$\begin{aligned}x_3 &= 480 - 15x_1 - 5x_2 \\ x_3 + 15x_1 &= 480 - 5x_2 \\ 15x_1 &= 480 - 5x_2 - x_3 \\ \frac{15}{15}x_1 &= \frac{480}{15} - \frac{5}{15}x_2 - \frac{1}{15}x_3 \\ x_1 &= 32 - \frac{1}{3}x_2 - \frac{1}{15}x_3\end{aligned}$$

Nun müssen wir noch das LP updaten mit der neuen Variable:

$$\begin{aligned} z &= 23(32 - \frac{1}{3}x_2 - \frac{1}{15}x_3) + 13x_2 \\ &= 736 - \frac{23}{3}x_2 - \frac{23}{15}x_3 + 13x_2 \\ &= 736 - \frac{23}{15}x_3 + \frac{16}{3}x_2 \end{aligned}$$

$$x_1 = 32 - \frac{1}{3}x_2 - \frac{1}{15}x_3$$

$$\begin{aligned} x_4 &= 160 - 4(32 - \frac{1}{3}x_2 - \frac{1}{15}x_3) - 4x_2 \\ &= 160 - 128 + \frac{4}{3}x_2 + \frac{4}{15}x_3 - 4x_2 \\ &= 32 + \frac{4}{15}x_3 - \frac{8}{3}x_2 \end{aligned}$$

$$\begin{aligned} x_5 &= 1190 - 20(32 - \frac{1}{3}x_2 - \frac{1}{15}x_3) - 35x_2 \\ &= 1190 - 640 + \frac{20}{3}x_2 + \frac{20}{15}x_3 - 35x_2 \\ &= 550 + \frac{20}{3}x_2 + \frac{20}{15}x_3 - \frac{105}{3}x_2 \\ &= 550 + \frac{20}{15}x_3 - \frac{85}{3}x_2 \\ &= 550 + \frac{4}{3}x_3 - \frac{85}{3}x_2 \end{aligned}$$

Das LP sieht nach dem Pivotschritt also folgendermaßen aus:

$$\begin{aligned} \max \quad & z = 736 - \frac{23}{15}x_3 + \frac{16}{3}x_2 \\ \text{mit} \quad & x_1 = 32 - \frac{1}{3}x_2 - \frac{1}{15}x_3 \\ & x_4 = 32 + \frac{4}{15}x_3 - \frac{8}{3}x_2 \\ & x_5 = 550 + \frac{4}{3}x_3 - \frac{85}{3}x_2 \end{aligned}$$

a.

$$\begin{aligned} \max \quad & z = 42 - 5x_2 + x_3 \\ \text{so dass} \quad & x_1 = 1 - 2x_2 - 2x_3 \\ & x_4 = 3 - 5x_2 + 7x_3 \\ & x_5 = 8 - 3x_2 - 2x_3 \\ & x_1, x_2, x_3, x_4, x_5 \geq 0 \end{aligned}$$

Figure 5: *Probeklausur Algorithmen der Informatik SoSe 2024 2024*

b.

$$\begin{aligned}
\max z &= 42 - 2x_2 + 5x_3 \\
\text{so dass } x_1 &= 2 - 2x_2 - 5x_3 \\
x_4 &= 2 - 5x_2 - 2x_3 \\
x_5 &= 7 - 3x_2 - x_3 \\
x_1, x_2, x_3, x_4, x_5 &\geq 0
\end{aligned}$$

Figure 6: *Klausur Algorithmen der Informatik Sommersemester 2021 2021*

c. *Probeklausur Algorithmen der Informatik SS 2021 2021*

$$\begin{aligned}
\text{maximiere } z &= 2x_1 - 3x_2 \\
\text{so dass } x_3 &= 2 - x_1 + x_2 \\
x_4 &= 3 + 2x_1 - 2x_2 \\
x_1, x_2, x_3, x_4 &\geq 0
\end{aligned}$$

Die Eingangsvariable könnten x_1 oder x_2 sein, da:

$$z = 2x_1 + 3x_2$$

Wir wählen x_1 als Entering-Variable.

Jetzt müssen wir berechnen was der Wert (die Ratio) von dem Koeffizienten der Entering-Variable durch das jeweilige "b" der Gleichung ist.

$$\begin{aligned}
x_3 &= 2 - 1x_1 + x_2 \\
x_4 &= 3 + 2x_1 - 2x_2
\end{aligned}$$

Die Ratios sind:

$$\begin{aligned}
\frac{2}{1} &= 0.5 && \text{für } x_3 \\
\frac{3}{2} &\approx 0.6 && \text{für } x_4
\end{aligned}$$

Die Leaving-Variable wird die mit der kleinsten Ratio, also x_3 .

Nun formen wir die zur Leave-Variable (x_3) gehörende Gleichung nach der Entering-Variable (x_1) um.

$$\begin{aligned}
x_3 &= 2 - x_1 + x_2 \\
x_1 + x_3 &= 2 + x_2 \\
x_1 &= 2 + x_2 - x_3
\end{aligned}$$

Nun müssen wir noch das LP updaten mit der neuen Variable:

$$\begin{aligned}
z &= 2x_1 - 3x_2 \\
&= 2(2 + x_2 - x_3) - 3x_2 \\
&= 4 + 2x_2 - 2x_3 - 3x_2 \\
&= 4 - 2x_3 - x_2
\end{aligned}$$

$$\begin{aligned}
x_4 &= 3 + 2(2 + x_2 - x_3) - 2x_2 \\
&= 3 + 4 + 2x_2 - 2x_3 - 2x_2 \\
&= 7 - 2x_3
\end{aligned}$$

Das LP sieht nach dem Pivotschritt also folgendermaßen aus:

$$\begin{array}{ll} \text{maximiere} & z = 4 - 2x_3 - x_2 \\ \text{so dass} & x_1 = 2 + x_2 - x_3 \\ & x_4 = 7 - 2x_3 \\ & x_1, x_2, x_3, x_4 \geq 0 \end{array}$$

5.4 Formen Sie folgendes LPaux um, so dass es eine gültige Basislösung hat und geben Sie diese an!

Besitzt die Grundform des LPs eine Lösung? Warum/Warum nicht? Falls Sie es nach einem Pivotschritt nicht beurteilen können, geben Sie einen Grund an.

a.

$$\begin{aligned}\max z &= -x_0 \\ \text{so dass } x_3 &= -x_1 + x_0 - 3 \\ x_4 &= 4x_2 + x_0 - 2 \\ x_1, x_2, x_3, x_4, x_0 &\geq 0\end{aligned}$$

Figure 7: *Probeklausur Algorithmen der Informatik SoSe 2024 2024*

b.

$$\begin{aligned}\max z &= -x_0 \\ \text{so dass } x_3 &= 2x_2 + x_0 \\ x_4 &= 4x_1 + x_0 - 3 \\ x_1, x_2, x_3, x_4, x_0 &\geq 0\end{aligned}$$

Figure 8: *Klausur Algorithmen der Informatik Sommersemester 2021 2021*

5.5 Besitzt folgendes LP eine zulässige Lösung? Warum / warum nicht?

a.

$$\begin{array}{ll}\max & x_1 - 42x_2 \\ \text{so dass} & x_3 \leq x_1 + x_2 + 2 \\ & x_1 + 2x_2 \leq -3x_3 \\ & x_1, x_2, x_3 \geq 0\end{array}$$

Figure 9: *Probeklausur Algorithmen der Informatik SS 2021 2021*

6 Multiple Choice

	Richtig	Falsch	
<i>Probeklausur</i> ☐ <i>Algorithmen der Informatik SS 2021</i> <i>2021 Klausur</i> <i>Algorithmen der Informatik Sommersemester 2021</i> <i>2021 Probeklausur</i> <i>Algorithmen der Informatik SoSe 2024</i> <i>2024</i>		☒	In einem Flussnetzwerk ist für einen gültigen Fluss die Kapazität jedes Schnittes (Cuts) gleich.
<i>Probeklausur</i> ☐ <i>Algorithmen der Informatik SS 2021</i> <i>2021 Probeklausur</i> <i>Algorithmen der Informatik SoSe 2024</i> <i>2024</i>		☒	Der Simplex-Algorithmus kann mit vier möglichen Ausgängen enden: Optimale Lösung, LP ungültig, LP unbeschränkt oder lokales, aber nicht globales Optimum des LPs.
<i>Nachklausur</i> ☒ <i>Algorithmen der Informatik SS 2024</i> <i>2024</i>		☐	Wenn die Basislösung des LP ungültig ist, besitzt das LP keine zulässige Lösung.

Müssen noch Fragen vom Spickzettel einfügen und Laufzeiten und wie man die Algorithmen anwendet. Algorithmen mit Problemen zusammenführen.

	Richtig	Falsch	
			Wir betrachten einen gerichteten Graphen und einen Startknoten. Wir nehmen an, dass einige Kantengewichte negativ sind, es aber keine Zyklen in dem Graphen gibt. Wenn wir den Dijkstra-Algorithmus anwenden...
<i>Probeklausur</i> <i>Algorithmen der Informatik SoSe 2024</i> <i>2024</i> <i>Probeklausur</i> <i>Algorithmen der Informatik SS 2021</i> <i>2021</i>	☐	☒	...könnte der Algorithmus in eine Endlosschleife geraten.
<i>Probeklausur</i> <i>Algorithmen der Informatik SoSe 2024</i> <i>2024</i>	☒	☐	...wird der Algorithmus garantiert terminieren, aber in einigen Fällen kann es passieren, dass die kürzesten Pfade nicht alle korrekt sind.
<i>Probeklausur</i> <i>Algorithmen der Informatik SS 2021</i> <i>2021</i>	☒	☐	Die Greedy Idee, welche der Dijkstra Algorithmus anwendet, kann nicht auf einem Graphen mit negativen Kantengewichten angewendet werden!
<i>Probeklausur</i> <i>Algorithmen der Informatik SoSe 2024</i> <i>2024</i> <i>Probeklausur</i> <i>Algorithmen</i>	☐	☒	...wird der Algorithmus garantiert terminieren und in einigen Fällen sind tatsächlich die kürzesten Pfade alle korrekt.

Index

- Adjazenzliste, 7, 11, 16
- Adjazenzmatrix, 5–8
- Arbitrage, 18
- Array, ungeordnet, 11, 14
- Aufwand
 - asymptotisch, 8
 - Kante hinzufügen, 5
 - Kante löschen, 5
 - Nachfolger finden, 6
 - Speicher-, 6
- Augmentierung, 32, 33
- Basislösung, 37, 38, 45, 49
- Basisvariable, 38
- Basisvariablen, 37, 40
 - Nicht-, 38, 40
- Bellman-Ford-Algorithmus, 18
- Boolesche Variable, 25, 26
- Boolesche Formel, 24
- Breitensuche, 13, 32
- Cut, 49
 - Minimum, 33
- Decrease-Key, 11
- Dijkstra, 6, 11, 12, 16, 20, 34, 51
- Distanzmatrix, 27, 32
- Dynamische Programmierung, 30
- Edmond-Karp Algorithmus, 32, 34
- Entering-Variable, 41, 43
- Eulerkreis, 4, 9, 10, 32, 34
- Eulerkreisalgorithmus, 9, 10
- Eulerpfad, 32, 34
- Extract-Min, 11
- FIFO-Prinzip, 13
- Floyd Algorithmus, 7, 8, 32
- Floyd-Warshall, 8, 27
- Fluss, 32, 33, 49
 - maximaler, 33, 34
 - minimaler, 49
 - optimal, 33
- Flussgraph, 49
- Flussnetzwerk, 32–34, 49
- Funktion
 - nichtlineare, 49
- Greedy, 30
- Grundform, 45
- Hash-Tabelle, 14, 15
- Heap, 14, 15
 - binärer, 11, 14–16
 - Fibonacci, 14–16
 - Min-, 14
- Heapsort, 12
- Hierholzer-Algorithmus, 9
- ILP, 22–26
- implex-Algorithmus, 49
- Iterative Verbesserung, 30
- Kantengewicht, 18
 - negatives, 18, 51
- Kantenliste, 5, 6, 8, 10
- Kapazität
 - residual, 32, 33
- Kruskal, 30, 34
- Laufzeit, 6, 14, 15
 - asymptotisch, 6
 - polynomielle, 49
- Leaving-Variable, 41, 43
- LP, 22, 29, 37, 38, 41, 42, 46, 49
- LPaux, 45
- Lösung
 - optimale, 49
- max-flow-min-cut, 33
- Nebenbedingungen, 36, 49
- Netzwerk
 - residual, 32
- Normalform
 - konjunktive, 25, 26, 49
- NP schwer, 49
- NP-hart, 49
- Optimum
 - globales, 49
- Pfad
 - augmentierend, 32
 - azyklischer, 51
 - kürzeste, 51
- Pivotkoeffizient, 43
- Pivotschritt, 41, 42, 44, 45
- Prim, 4, 12, 51
- Programmierparadigmen, 30
- Queue, 13
 - Priority, 11, 12, 14, 16
 - Indexed, 12, 15
- Relaxation, 22–24
- SAT, 24
 - MAXSAT, 24–26
- Schlupf-Form, 38
- Schlupfform, 37, 40
- Schlupfvariable, 36
- Spannbaum
 - Produkt-, 51
- Standardform, 36, 40

Tiefensuche, [4](#)

Vertex Cover, [28](#)
 Weighted, [29](#)

Vertex Cover, Weighted, [28](#)

Wert
 optimaler, [24](#)

Zielfunktion, [36–39](#)

List of Algorithms

1	Hierholzer-Algorithmus für einen Eulerkreis	9
2	Approximation des Weighted Vertex Cover Problems	28
3	Weighted Vertex Cover	28
4	Simplex-Verfahren	41

References

- Klausur Algorithmen der Informatik Sommersemester 2021* (2021). Interne Ressource, nicht öffentlich verfügbar.
- Nachklausur Algorithmen der Informatik SS 2019* (2019). Interne Ressource, nicht öffentlich verfügbar.
- OpenAI (2024). *ChatGPT*. Large language model. URL: <https://www.openai.com/chatgpt>.
- Probeklausur Algorithmen der Informatik SoSe 2024* (2024). Interne Ressource, nicht öffentlich verfügbar.
- Probeklausur Algorithmen der Informatik SS 2021* (2021). Interne Ressource, nicht öffentlich verfügbar.
- Schulz, A. (2024). *Algorithmen der Informatik: Lineare Optimierung*. Online. Vorlesungsfolien, Teil 5.
- Wikipedia contributors (2024). *Max-flow min-cut theorem* — *Wikipedia, The Free Encyclopedia*. [Online; accessed 16-September-2024]. URL: https://en.wikipedia.org/w/index.php?title=Max-flow_min-cut_theorem&oldid=1244684321.